Build SmartPOS Yourself

A 5-day plan for a beginner

4 hours a day · 20 hours in total · every module, file and name you need, in the order to build them, with a check at the end of every block.

You will build: a point-of-sale web app with sign-up and login, products and stock, customers with loyalty points, a cashier screen that sells safely, bills, and a small dashboard.
Stack: ASP.NET Core 10 · EF Core · PostgreSQL · Angular. The finished reference version is in this repository, so you can compare your work with it.

1. Read this first

What this plan is, and is not

This is a **build plan**, not a tutorial that gives you the code. For each module it tells you the file names, what the file must do, a few hints, and a check that proves it works. **You write the code.** The finished project in this repository is your answer key: open the matching file *after* you have tried it yourself (or after 15 minutes stuck), compare, and learn from the difference.

Be realistic. The finished SmartPOS is several weeks of professional work. Twenty hours is enough for a smaller version that is still real and useful: it sells, deducts stock safely, remembers customers and gives them points. Section 3 lists exactly what is cut, and each day has a 'if you fall behind' list so you always finish with something that works rather than ten half-built things.

The rules that make 4 hours a day work

- **Type the code, never paste it.** Typing is how names and syntax stick. Copy only commands.
- **One block at a time, then a check.** Do not start the next block until the 'Done when' check passes.
- **Commit after every block** (`git add . && git commit -m "day 2 block 3: products"`). If you break something, you can go back.
- **Stuck for 15 minutes?** Read the error message slowly, search for its first line, then look at the reference file. Do not stay stuck for an hour.
- **Keep a notes file** (`NOTES.md`): one line per day for 'what I learned' and 'what confused me'. Day 5 uses it.
- **Leave 10 minutes at the start of each day** to rerun yesterday's checks. A day that starts green ends green.

How each 4-hour day is split

When	What
0:00 - 0:10	Warm-up: start the database and the app, rerun yesterday's last check.
Four blocks of about 55 minutes	Build. Each block ends with a 'Done when' check and a git commit.
Three 5-minute breaks	Between blocks. Stand up; do not skip them.

Before day 1: install and check (do this the evening before)

Tool	How to check
.NET 10 SDK	`dotnet --version` prints 10.x
Node.js 24 and npm	`node --version` prints v24.x
PostgreSQL 16	`psql --version` works, and the server is running
Git	`git --version`
VS Code with the C# Dev Kit and Angular Language Service extensions	Open any .cs and .ts file: you should see colours and error squiggles

Tip: on this machine the .NET SDK lives in `~/.dotnet` and may not be on your PATH; if `dotnet` is 'not found', run `export PATH="$HOME/.dotnet:$PATH"` in that terminal.

2. The five days at a glance

Day	Outcome by the end of the day
Day 1	**Foundation and database.** Solution skeleton, the 7 entities, the database context, the first migration, tables in PostgreSQL.
Day 2	**Backend: errors, login, products.** Error handling, password hashing, JWT login, product search/create/update through the API.
Day 3	**Backend: selling.** The sale calculator (test-first), safe stock deduction, the sale service, inventory, customers and loyalty points.
Day 4	**Frontend.** Angular app, login and sign-up, layout, products pages, the cashier (POS) screen.
Day 5	**Finish.** Customers and bills in the UI, a simple dashboard, an integration test, security checks, notes and a stretch choice.

The modules you will build, in order

#	Module	Contains	When
1	**SmartPOS.Core**	Entities and enums	Day 1
2	**SmartPOS.Infrastructure / Persistence**	DbContext, entity configurations, migrations	Day 1
3	**SmartPOS.Application / Common**	Errors, paging, the DbContext interface, options	Day 2
4	**Auth** (Application + Infrastructure + API)	Register, login, password hashing, JWT	Day 2
5	**Products**	Service, DTOs, controller	Day 2
6	**Sales**	SaleCalculator, SaleService, SalesController	Day 3
7	**Inventory**	StockRules, atomic stock update, adjustments	Day 3
8	**Customers + Loyalty**	CustomerService, LoyaltyRules	Day 3 / 5
9	**Frontend core**	models, auth service, interceptor, guards, routes	Day 4
10	**Frontend layout + shared**	shell, sidebar, header, states, money pipe	Day 4
11	**Frontend features**	login, register, products, pos, customers, sales	Day 4 / 5
12	**Dashboard**	summary endpoint, one chart	Day 5
13	**Tests**	unit tests (calculator, loyalty, cart), one integration test	Days 3 / 4 / 5

3. What you will build, and what is cut

The target: the 'learner edition'

```
Browser (Angular)  --HTTP/JSON-->  ASP.NET Core API  -->  EF Core  -->  PostgreSQL
login | products | POS | customers | bills | dashboard
```

Seven tables: **Users, Products, Inventories, Customers, Sales, SaleItems, StockMovements**. One shop, one kind of user. Prices are always decided by the server, stock can never go below zero, and every stock change is written to a ledger.

Deliberately cut (the finished app has these; you can add them later)

Cut	Why / when to add it
Multiple shops (ShopId everywhere + query filters)	Biggest source of extra work. It is Stretch A in Section 11. Build single-shop first; adding it later teaches you filters.
Email verification and email checks	Needs a mail sender and a token flow. Stretch B.
Demo-history generator, backfill and password-reset commands	Developer tools, not product features. Stretch D.
The full insights dashboard (heatmap, donut, KPI comparisons)	You build one simple chart on Day 5. The full set is Stretch C.
Reports page, barcode field, expiry dates, categories endpoint	Nice to have. Products keep a plain Category text field.
Line-level discount and tax allocation	Your calculator works at bill level; Section 7 shows the simple version and what the finished one adds.
Rate limiting, health checks, CORS polish	You add CORS and login rate limiting on Day 5 if time allows.

Names you must use

Keep these names exactly. They are the names in the reference project, so error messages, snippets in this guide and the answer key all line up. Folders: `backend/src/SmartPOS.Core`, `SmartPOS.Application`, `SmartPOS.Infrastructure`, `SmartPOS.API`, `backend/tests/SmartPOS.UnitTests`, `frontend/`.

4. Just-enough theory

Read this section once (20 minutes), then come back to it when a word confuses you. Each idea appears again in the day where you use it.

Layers: why four projects?

```
API  ->  Infrastructure  ->  Application  ->  Core          (an arrow means 'may use')
```

Layer	Holds
Core	The things your shop talks about: Product, Sale, Customer. Plain classes. Knows nothing about databases or web.
Application	The rules: how a sale is priced, when stock is too low, how points are earned. Pure logic, easy to test.
Infrastructure	The outside world: PostgreSQL through EF Core, password hashing.
API	The front door: HTTP controllers, login tokens, error responses.

Why bother? You can test the sale rules without a database or a web server, and you can swap the database without touching the rules.

HTTP, REST and JSON in one table

Request	Meaning
GET /api/products	Read a list. Never changes anything.
POST /api/products	Create. The body is JSON, for example {"sku":"A1","name":"Tea"}.
PUT /api/products/5	Replace product 5.
DELETE /api/products/5	Remove product 5.

Status	Meaning
200 / 201 / 204	OK / Created / OK with nothing to return
400	Your request is wrong (validation failed, or a business rule such as 'cart is empty')
401 / 403	Not signed in / signed in but not allowed
404 / 409	Does not exist / conflicts with existing data (duplicate SKU, not enough stock)
500	A bug on the server

Databases and EF Core

- **Table** = rows and columns. **Primary key** = the unique id of a row. **Foreign key** = a column that points at another table's row.
- **Unique index**: the database refuses duplicates (two products with one SKU). **CHECK constraint**: the database refuses bad values (negative stock). These are your last line of defence if your code has a bug.
- **Transaction**: a group of changes that all happen or none do. A sale changes stock *and* inserts the bill: it must be one transaction.
- **EF Core** turns C# classes into tables. A **DbContext** is your connection; a **DbSet<Product>** is the Products table; LINQ (`db.Products.Where(...)`) becomes SQL.
- **Migration**: a versioned script that creates or changes tables. You never edit tables by hand; you change C#, add a migration, apply it.

Money is never a double

Computers store `double` numbers in binary, so 0.1 + 0.2 is 0.30000000000000004. Use `decimal` in C# and `numeric(18,2)` in PostgreSQL. In the browser, do money maths in whole cents (integers) and divide by 100 only to display. Round with `Math.Round(x, 2, MidpointRounding.AwayFromZero)`, which is what people expect on a receipt.

Passwords and tokens

- **Hashing is not encryption.** A hash cannot be reversed. You store the hash, and at login you hash what was typed and compare. Use a proven hasher; never invent your own.
- **JWT** (JSON Web Token): after login the server gives the browser a signed string containing the user id. The browser sends it in a header on every request (`Authorization: Bearer ...`). The server checks the signature, so it does not need to remember sessions.
- **Secrets stay out of git.** The database password and the token signing key go in `dotnet user-secrets`, never in a file you commit.

Angular in one page

Word	Meaning
Component	A piece of screen: a TypeScript class plus an HTML template. Example: ProductList.
Signal	A value the screen watches: `count = signal(0)`; read `count()`; change `count.set(1)`. When it changes, the screen updates.
Service	A class that holds logic or talks to the API (ProductsService). Components ask for it with `inject(ProductsService)`.
Route	Maps a URL to a component: /products shows ProductList. Routes can be lazy-loaded.
Interceptor	Code that runs on every HTTP call: yours adds the login token.
Guard	Blocks a route unless a condition holds (signed in).
Reactive form	A form described in code with validators (required, min length); the template shows errors.

Two kinds of test

A **unit test** checks one piece of logic in isolation (the sale calculator). An **integration test** starts the real API in memory and calls it like a browser would. You write unit tests while building (they are fast), and one integration test on Day 5.

5. Day 1: foundation and database

Goal: an empty solution that builds, seven entities, and the tables created in a real PostgreSQL database. **You will learn:** project layout, entities, EF Core, migrations.

Block	What you build (names to use)	Done when
1 0:10-1:05	**Scaffold the solution.** Folders backend/ and frontend/; solution **SmartPOS**; projects **SmartPOS.Core**, **SmartPOS.Application**, **SmartPOS.Infrastructure** (class libraries), **SmartPOS.API** (web API with controllers), **SmartPOS.UnitTests** (xUnit). Set the references to follow the arrows in Section 4.	`dotnet build` succeeds. `git log` shows your first commit. A .gitignore exists (bin, obj, node_modules).
2 1:10-2:05	**Core entities and enums** in SmartPOS.Core: **User, Product, Inventory, Customer, Sale, SaleItem, StockMovement**; enums **PaymentMethod** (Cash, Card, UPI, Other), **SaleStatus** (Pending, Completed, Cancelled), **StockMovementType** (InitialStock, Sale, Adjustment).	The solution builds. You can say, for each entity, what one row means and which other entity it points at.
3 2:10-3:05	**SmartPosDbContext and configurations** in Infrastructure/Persistence: one class per entity in Configurations/ (lengths, decimal precision, unique indexes, CHECK constraints, enum stored as text). Connection string via user-secrets.	`dotnet build` passes with the Npgsql package installed; user-secrets list shows your connection string.
4 3:10-4:00	**First migration** InitialCreate; create the PostgreSQL role and database; apply the migration; look at the tables.	`\dt` in psql lists 7 tables plus __EFMigrationsHistory, and inserting a product with negative stock is rejected.

Block 1 commands

```
mkdir smartpos && cd smartpos && git init
mkdir backend frontend && cd backend
dotnet new sln -n SmartPOS
dotnet new classlib -n SmartPOS.Core           -o src/SmartPOS.Core
dotnet new classlib -n SmartPOS.Application    -o src/SmartPOS.Application
dotnet new classlib -n SmartPOS.Infrastructure -o src/SmartPOS.Infrastructure
dotnet new webapi   -n SmartPOS.API            -o src/SmartPOS.API --use-controllers
dotnet new xunit    -n SmartPOS.UnitTests      -o tests/SmartPOS.UnitTests
dotnet sln add src/SmartPOS.Core src/SmartPOS.Application src/SmartPOS.Infrastructure
dotnet sln add src/SmartPOS.API tests/SmartPOS.UnitTests
dotnet add src/SmartPOS.Application reference src/SmartPOS.Core
dotnet add src/SmartPOS.Infrastructure reference src/SmartPOS.Application
dotnet add src/SmartPOS.API reference src/SmartPOS.Infrastructure
dotnet add tests/SmartPOS.UnitTests reference src/SmartPOS.Application
```

If a reference command creates a circular reference, you pointed an arrow the wrong way. Core references nothing.

Two clean-ups in the API project (tested)

- **Delete the template's sample:** remove `Controllers/WeatherForecastController.cs` and `WeatherForecast.cs` from SmartPOS.API.
- **Fix the port.** The template picks a random port (for example 5173). Open `src/SmartPOS.API/Properties/launchSettings.json` and set the **http** profile's `applicationUrl` to `http://localhost:5080`. This guide, the Angular app and the check scripts all assume 5080.

Packages: install each when you first need it

```
cd backend
dotnet add src/SmartPOS.Application    package Microsoft.EntityFrameworkCore
dotnet add src/SmartPOS.Application    package Microsoft.Extensions.Options.DataAnnotations
dotnet add src/SmartPOS.Infrastructure package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/SmartPOS.Infrastructure package Microsoft.Extensions.Identity.Core
dotnet add src/SmartPOS.API            package Microsoft.EntityFrameworkCore.Design
dotnet add src/SmartPOS.API            package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
dotnet add src/SmartPOS.API          package Swashbuckle.AspNetCore
```

Package	Why
Microsoft.EntityFrameworkCore (Application)	Lets the Application layer use DbSet and LINQ queries (IApplicationDbContext).
Npgsql.EntityFrameworkCore.PostgreSQL (Infrastructure)	The PostgreSQL provider for EF Core.
Microsoft.Extensions.Identity.Core (Infrastructure)	The password hasher (Day 2).
Microsoft.EntityFrameworkCore.Design (API)	Needed by the dotnet ef tool to create migrations.
Microsoft.AspNetCore.Authentication.JwtBearer (API)	Checks the JWT on each request (Day 2).
Swashbuckle.AspNetCore (API)	The Swagger page you will use all week. The .NET 10 template only produces the raw OpenAPI file, not the page.

These commands were tested on a fresh solution and build with 0 warnings. Versions will be newer than the reference's (10.0.14 vs 10.0.12); that is fine.

Block 2: what each entity needs (compare with the reference: backend/src/SmartPOS.Core/Entities/)

Entity	Fields
User	Id, FullName, Email (store lower-case), PasswordHash, CreatedAt, UpdatedAt, LastLoginAt
Product	Id, Sku, Name, Description?, Category, Price, CostPrice, IsActive, CreatedAt, UpdatedAt, **Inventory?** (navigation)
Inventory	Id, ProductId, **Product**, StockQuantity, LowStockThreshold (default 20), LastUpdated. One row per product.
Customer	Id, Name, Phone, LoyaltyPoints, CreatedAt
Sale	Id, BillNumber, UserId, **User**, CustomerId?, **Customer?**, Subtotal, DiscountPercent, DiscountAmount, TaxRatePercent, TaxAmount, TotalAmount, PaymentMethod, AmountPaid, ChangeAmount, LoyaltyPointsEarned, Status, CreatedAt, **Items** (list)
SaleItem	Id, SaleId, ProductId, ProductName, Sku, Quantity, UnitPrice, UnitCost, DiscountAmount, TaxAmount, LineTotal
StockMovement	Id, ProductId, QuantityChange (+in, -out), QuantityAfter, Type, Reason, SaleId?, UserId?, CreatedAt

A **?** means optional (nullable). Fields shown in bold are *navigation properties*: they let you write `sale.Customer.Name` instead of joining by hand.

Why: SaleItem copies the product's name, SKU, price and cost instead of just pointing at the product. If the price changes next month, last month's bills must not change. This is called a snapshot.

```
// Hint: the shape of one entity (write the others the same way)
public class Product
{
    public int Id { get; set; }
    public string Sku { get; set; } = string.Empty;
    public decimal Price { get; set; }          // decimal, never double
    public Inventory? Inventory { get; set; }   // navigation: optional single related row
}
```

Block 3: configuration checklist

- [ ] Every string has a maximum length (`HasMaxLength(150)`) and required strings are `IsRequired()`.
- [ ] Money columns use `HasPrecision(18, 2)`. Percent columns use (5, 2).
- [ ] Unique indexes: User.Email, Product.Sku, Sale.BillNumber, Customer.Phone, Inventory.ProductId.
- [ ] Enum properties stored as text: `HasConversion<string>()` so the database shows 'Cash', not 1.

- [ ] CHECK constraints: stock >= 0, prices >= 0, quantity > 0, QuantityAfter >= 0. **In PostgreSQL the column names inside a CHECK must be quoted**: `"\"Price\" >= 0"` in C#, otherwise PostgreSQL lower-cases the name and the migration fails.
- [ ] Delete rules: a SaleItem's product uses **Restrict** (a sold product can never be deleted); a Sale's items use **Cascade**; Inventory cascades from its product.
- [ ] In your DbContext, expose DbSets for all 7 entities and call `ApplyConfigurationsFromAssembly`.

```
cd src/SmartPOS.API
dotnet user-secrets init
CS="Host=127.0.0.1;Port=5432;Database=smartpos_dev;Username=smartpos;Password=YOUR-PASSWORD"
dotnet user-secrets set "ConnectionStrings:SmartPos" "$CS"
```

Block 4: create the database and the tables

```
psql postgres -c "CREATE ROLE smartpos LOGIN PASSWORD 'YOUR-PASSWORD'"
psql postgres -c "CREATE DATABASE smartpos_dev OWNER smartpos"

cd backend
dotnet new tool-manifest
dotnet tool install dotnet-ef
dotnet add src/SmartPOS.API package Microsoft.EntityFrameworkCore.Design
dotnet ef migrations add InitialCreate --project src/SmartPOS.Infrastructure \
    --startup-project src/SmartPOS.API --output-dir Persistence/Migrations
dotnet ef database update --project src/SmartPOS.Infrastructure --startup-project src/SmartPOS.API

psql smartpos_dev -c "\dt"
psql smartpos_dev -c "\d \"Products\""
```

Experiment (5 minutes, important): in psql, try `INSERT INTO "Inventories" ("ProductId", "StockQuantity", ...) VALUES (1, -5, ...)`. Read the error. That is a CHECK constraint protecting your data even from your own bugs.

Day 1 self-check (answer out loud)

- Why does Core not reference any other project?
- What is the difference between a migration and the database itself?
- Why is Price a decimal and not a double?
- Why does SaleItem repeat the product name?

If you fall behind on Day 1

Drop **Customer** and the **LoyaltyPointsEarned** column from this day (add them on Day 3, with a second migration, which is good practice anyway). Never drop StockMovement or the CHECK constraints: they are what make the later days safe.

6. Day 2: errors, login, products

Goal: from Swagger you can sign up, sign in and manage products with proper validation and error messages.
You will learn: services and controllers, DTOs, validation, hashing, JWT.

Block	What you build (names to use)	Done when
1	**Application/Common: AppExceptions** (BusinessRuleException, NotFoundException, ConflictException, ForbiddenException, AuthenticationFailedException), **Paging** (PagedQuery, PagedResult<T>), **IApplicationDbContext** (DbSets + SaveChangesAsync + BeginTransactionAsync). In the API: **GlobalExceptionHandler** turning each exception into the right status code.	A controller action that throws NotFoundException returns 404 with a JSON body, not a crash page.
2	**Auth.** Application/Auth: **IPasswordHasher**, **ITokenService**, **AuthDtos** (LoginRequest, RegisterRequest, LoginResponse), **IAuthService** + **AuthService**. Infrastructure/Security: **IdentityPasswordHasher**. API/Auth: **JwtOptions**, **JwtTokenService**. API/Controllers: **AuthController** (register, login).	Register then login in Swagger returns an accessToken. A wrong password returns 401 with the *same* message as an unknown email.
3	**JWT in Program.cs** (AddAuthentication + AddJwtBearer, signing key from user-secrets, validate issuer, audience, lifetime) and a **fallback policy** that requires login everywhere. Then **Products: ProductDtos**, **IProductService** + **ProductService**, **ProductsController**.	Without a token GET /api/products is 401. With the token (Swagger's Authorize button) you can create, list, search and page products.
4	**Product rules and polish:** SKU upper-cased and unique (409 on duplicate), validation attributes on the DTOs, delete only if never sold, creating a product also creates its **Inventory** row and an InitialStock **StockMovement** in one SaveChanges.	Duplicate SKU gives 409; empty name gives 400 listing the field; a new product with stock 10 shows 10 in Inventories and 1 row in StockMovements.

Block 1: the exception handler idea

```
// AppExceptions.cs: one base type, one subclass per HTTP meaning
public abstract class AppException(string message) : Exception(message);
public sealed class NotFoundException(string message) : AppException(message);
public sealed class ConflictException(string message) : AppException(message);

// GlobalExceptionHandler.cs (API): map type -> status code, write a ProblemDetails JSON body.
// Anything that is NOT an AppException: log it, return a generic 500 (never leak details).
```

Why exceptions for business errors? A service can say `throw new ConflictException("SKU already exists")` from deep inside, and the handler turns it into a clean 409 in one place.

Block 2: auth hints

- **Registration:** lower-case and trim the email; check it is unused; hash the password; save; return a token.
- **Login:** find the user; verify the hash; on any failure throw the *same* AuthenticationFailedException (never say which part was wrong). For an unknown email, still run a dummy hash check so the response takes about as long.
- **Password rule:** 8+ characters with a letter and a digit (a regular expression on the request DTO).
- **Hasher:** install `Microsoft.Extensions.Identity.Core` and wrap its `PasswordHasher<T>` behind your own IPasswordHasher interface.
- **Token claims:** sub (user id), name, email, jti (random id). Expiry 8 hours. Sign with HMAC-SHA256 using a key of at least 32 characters from user-secrets (`dotnet user-secrets set "Jwt:SigningKey" "$(openssl rand -base64 48)"`).

Block 3: Swagger with a Bearer token (tested snippet)

Swagger only appears in the **Development** environment, which the template's launch profile already sets. Without the security part Swagger can't send your token, and every call looks like a 401.

```
using Microsoft.OpenApi;                        // top of Program.cs

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
        { Type = SecuritySchemeType.Http, Scheme = "bearer", BearerFormat = "JWT" });
    options.AddSecurityRequirement(document => new OpenApiSecurityRequirement
        { [new OpenApiSecuritySchemeReference("Bearer", document)] = [] });
});

if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }   // then open /swagger
```

Remove the template's `AddOpenApi()` and `MapOpenApi()` lines to avoid confusion. Click **Authorize** in Swagger and paste the accessToken (without the word Bearer).

Block 3: wiring Program.cs (in this order)

- [ ] AddControllers with a JSON string-enum converter (so enums appear as text).
- [ ] AddExceptionHandler<GlobalExceptionHandler> and AddProblemDetails; UseExceptionHandler().
- [ ] AddDbContext<SmartPosDbContext>(UseNpgsql(connection string)) and map IApplicationDbContext to it.
- [ ] Register your services (AddScoped for each service, AddSingleton for the hasher and token service).
- [ ] AddAuthentication().AddJwtBearer(...), AddAuthorization with the fallback policy, then UseAuthentication() *before* UseAuthorization().
- [ ] AddSwaggerGen with a Bearer security scheme so Swagger can send the token.
- [ ] [AllowAnonymous] only on register and login.

Block 4: product search

`GET /api/products?search=milk&page=2&pageSize=20`. Build the query in steps (`var q = db.Products.AsNoTracking();` then add `Where` clauses), then `Skip((page-1)*pageSize).Take(pageSize)` and a separate `CountAsync` for the total. **PostgreSQL's LIKE is case-sensitive**, so compare lower-cased values: `EF.Functions.Like(p.Name.ToLower(), pattern.ToLower())`. Escape `%` and `_` in what the user typed (reference: Common/SearchPattern.cs).

Day 2 self-check

- What is stored in the database for a password, and why can't it be turned back into the password?
- What does the server check when a request arrives with a JWT?
- Why should login errors be identical for 'no such user' and 'wrong password'?
- What does AsNoTracking do, and when is it safe?

If you fall behind on Day 2

Skip the product *update* and *delete* endpoints and keep list, search, create. Skip Swagger styling. Do **not** skip the exception handler or the password rules.

7. Day 3: selling (the heart of the system)

Goal: a sale request is priced by the server, deducts stock safely, records the bill and the stock ledger, and gives loyalty points. **You will learn:** test-first development, transactions, atomic updates. This is the hardest day; protect it.

Block	What you build (names to use)	Done when
1	**SaleCalculator** (Application/Sales), written **test-first**: write SaleCalculatorTests in SmartPOS.UnitTests, watch them fail, then make them pass. Also **Settle()** for cash change and **DiscountPolicy**.	`dotnet test` is green with at least the 8 cases below.
2	**Inventory. StockRules** (EnsureAvailable, EnsureValidAdjustment), **TryChangeStockAsync** in the DbContext, **InventoryService** (list, adjust with a reason, list movements) and **InventoryController**.	An adjustment of -1000 on stock 5 gives 409; +10 works and shows in the movements list.
3	**SaleService.CreateAsync**, **SaleDtos**, **ISaleService**, **SalesController** (create, list, get one).	A sale for 3 of a product with stock 10 returns the bill, stock becomes 7, one Sale + one SaleItem + one StockMovement exist.
4	**Customers + loyalty. CustomerService** + **CustomersController** (list/search, create, update), **LoyaltyRules**, and the points step inside SaleService. Run the **parallel-sales check**.	12 parallel sales on stock 5: exactly 5 succeed, stock ends at 0. A 456.75 bill for a customer adds 4 points.

Block 1: the arithmetic, and the tests to write first

Rules (the server is the only place that decides prices): line subtotal = unit price x quantity; subtotal = sum of lines; **discount** = subtotal x discount% (rounded); **tax** = (subtotal - discount) x tax% (rounded, so tax is charged *after* the discount); **total** = subtotal - discount + tax. Round every step to 2 decimals, half away from zero.

Test input	Expected result
Lines: 3 x 99.99 and 3 x 40.00; discount 10%; tax 5%	subtotal 419.97, discount 42.00, tax 18.90, total 396.87
One line 100.00 x 1; discount 20%; tax 10%	discount 20.00, tax 8.00 (10% of 80, not of 100), total 88.00
12.50 x 2, no discount, no tax	total 25.00
One line, discount 100%, tax 18%	total 0.00
Round(0.125), Round(0.135), Round(2.004)	0.13, 0.14, 2.00 (midpoints go away from zero, not to even)
Empty cart, quantity 0, negative price, discount -1 or 101	each throws BusinessRuleException
Settle(total 105, Cash, paid 200)	paid 200, change 95. Paid 100 throws (less than total).
Settle(total 50, UPI, paid 60)	throws: change can only be given for cash. Paid omitted means exactly the total.

Simplification for you: compute the discount and tax for the whole bill and store them on the Sale. For each SaleItem set `LineTotal = unit price x quantity` and leave the line's DiscountAmount and TaxAmount at 0. The finished app also splits discount and tax across lines to the cent (so the lines add up exactly); see Application/Sales/SaleCalculator.cs, method Allocate, when you want that (Stretch E).

Block 2: the stock update that cannot oversell

The classic bug: read stock (5), check it is enough, then write stock-3. Two cashiers can both read 5 and both sell the last 5 units. The fix is to make the *check and the write one SQL statement*:

```
// SQL idea:   UPDATE "Inventories" SET "StockQuantity" = "StockQuantity" + @delta
//             WHERE "ProductId" = @id AND "StockQuantity" >= -@delta
// If 0 rows changed, there was not enough stock -> return null and the sale is rolled back.
```

```
public async Task<int?> TryChangeStockAsync(int productId, int delta, CancellationToken ct)
{
    var affected = await Inventories
        .Where(i => i.ProductId == productId && i.StockQuantity >= -delta)
        .ExecuteUpdateAsync(s => s.SetProperty(i => i.StockQuantity, i => i.StockQuantity + delta), ct);
    if (affected == 0) return null;
    return await Inventories.Where(i => i.ProductId == productId)
                            .Select(i => i.StockQuantity).SingleAsync(ct);
}
```

Put the method on IApplicationDbContext so services can call it. The database also holds the row lock until your transaction ends, so the second cashier waits, then re-checks and fails cleanly.

Block 3: SaleService.CreateAsync, step by step

- [ ] Merge duplicate lines (the same product twice becomes one line) and **sort by product id** (a consistent order prevents deadlocks between two sales).
- [ ] Load the products **from the database** (never trust a price from the request) and reject unknown or inactive products with a 400.
- [ ] Check the discount limit; call SaleCalculator.Calculate; call Settle for paid and change.
- [ ] Build the Sale with a BillNumber such as `BILL-yyyyMMdd-` + 8 random hex characters.
- [ ] `await using var tx = await db.BeginTransactionAsync(ct);`
- [ ] For each line: call TryChangeStockAsync with a negative quantity; if null, throw ConflictException (the transaction is rolled back when it is disposed). Add the SaleItem (snapshot name, sku, price, cost) and a StockMovement (Type Sale, QuantityAfter from the call).
- [ ] Add the sale, SaveChangesAsync, then `await tx.CommitAsync(ct)`. Return the full bill.
- [ ] The request carries only: product ids, quantities, discount %, payment method, optional amount paid, optional customer id. **No prices, no totals.**

Block 4: loyalty points, and proving there is no overselling

Bill total (rate 100)	Points = floor(total / 100)
99.99	0
100.00	1
456.75	4
3,076.50	30
0 or negative	0

- Write `LoyaltyRules.PointsEarned(decimal total, decimal spendPerPoint)` test-first. Always round **down**; a rate of 0 or less earns 0.
- Read `Loyalty:SpendPerPoint` (default 100) from configuration with an options class.
- In SaleService, when a customer is attached, store the points on the Sale (`LoyaltyPointsEarned`) and add them to the customer's balance **with one atomic update** (`ExecuteUpdateAsync`, balance = balance + n) inside the same transaction. Do not read the balance, add in C#, and save: two bills at once would lose points.
- Walk-in bills (no customer) earn 0.

```
# In Swagger: log in, copy the accessToken, create a product with initial stock 5 (note its id).
TOKEN=paste-the-token-here
URL=http://localhost:5080/api/sales
AUTH="Authorization: Bearer $TOKEN"
JSON="Content-Type: application/json"
BODY='{"items":[{"productId":1,"quantity":1}]}'
seq 12 | xargs -P12 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST -H "$AUTH" -H "$JSON" -d "$BODY" $URL | sort | uniq -c
# Expected:  5 x 201   and   7 x 409
```

Day 3 self-check

- Why does the server calculate prices instead of trusting the browser?
- What would go wrong if stock were read in C#, checked, then written back?
- Why must the stock updates and the bill insert be in one transaction?
- Why round points down?

If you fall behind on Day 3

Move **customers and loyalty** to Day 5 (the sale works without them). Never skip the calculator tests or the atomic stock update.

8. Day 4: the Angular frontend

Goal: in the browser you can sign up, sign in, manage products, and complete a sale at the cashier screen. **You will learn:** components, signals, services, routing, forms, an HTTP interceptor.

Block	What you build (names to use)	Done when
1	**Scaffold and core.** App **smartpos-web** in frontend/. **environments** (apiUrl), **core/models.ts**, **core/auth/auth.service.ts**, **auth.interceptor.ts**, **auth.guards.ts**, **app.routes.ts** (lazy loadComponent), **features/auth/login** and **register**.	You can register and log in from the browser; the token is in localStorage; the dashboard route is blocked when signed out.
2	**Layout and shared. layout/shell**, **sidebar**, **header**; **shared/components/states** (loading, error, empty), **pagination**; **shared/pipes/money.pipe.ts**; **core/api/settings.service.ts**; a small **styles.css** with colour variables.	After login you see a sidebar and header around the page. Money shows as currency.
3	**Products. core/api/products.service.ts**, **features/products/product-list** (search box with a short delay, pagination) and **product-form** (reactive form with validators, create and edit).	You can add a product in the form and find it by searching; invalid input shows error messages.
4	**The cashier (POS). features/pos/cart.ts** (pure functions) with **cart.spec.ts**, then **pos.ts / pos.html**: product tiles, search, cart, discount, payment method, checkout.	You complete a sale in the browser and the stock of that product drops by the quantity sold.

Block 1: commands and the pieces

```
cd frontend
npx @angular/cli new smartpos-web --routing --style=css --ssr=false
cd smartpos-web && npm start          # http://localhost:4200  (leave it running)
```

- **CORS:** the browser (port 4200) calls the API (port 5080), so the API must allow that origin. Add a CORS policy in Program.cs with `WithOrigins("http://localhost:4200")`, and call UseCors before authentication. The error 'blocked by CORS policy' in the browser console means this is missing.
- **AuthService:** a `signal` holding the session (token + user), `login()` and `register()` returning the HTTP call and saving the session, `logout()`. Keep the session in localStorage so a refresh does not sign you out (wrap in try/catch).
- **Interceptor:** clone each request and add the `Authorization: Bearer` header; on a 401 response, log out and go to /login.
- **Guard:** a function that returns true if signed in, otherwise a redirect to /login. Attach it to the shell route that holds all signed-in pages.

Block 4: the cart as pure functions (test these first)

Do the maths in **whole cents** so 19.99 x 3 is exactly 5997, not 59.97000000000001. The server stays the authority; this is only a live preview that must match it.

```
// cart.ts - no Angular here, just functions you can test
export interface CartLine {
  productId: number; name: string; unitPrice: number; quantity: number; available: number;
}

// merge the same product, respect stock:
export function addProduct(lines: CartLine[], product: Product): CartLine[]
// clamp the quantity to 1..available:
export function setQuantity(lines: CartLine[], productId: number, qty: number): CartLine[]
export function removeLine(lines: CartLine[], productId: number): CartLine[]
// returns { subtotal, discountAmount, taxAmount, total }, same rules as the server:
export function calculateTotals(lines: CartLine[], discountPercent: number, taxPercent: number)
```

Write `cart.spec.ts` with the same example as Day 3 (3 x 99.99 and 3 x 40.00, 10% discount, 5% tax gives total 396.87) and an out-of-stock case. Run it with `npm test`.

- [ ] The POS component keeps its state in signals: `lines`, `discountPercent`, `paymentMethod`; the totals are a `computed()` signal, so they update automatically.
- [ ] Checkout sends only product ids and quantities (no prices) and shows the server's bill. Show the server's error message if it answers 409.
- [ ] After a successful sale, clear the cart and reload the product tiles so the stock numbers refresh.

Day 4 self-check

- What does the interceptor do, and why is it better than adding the header in every service?
- Why do the cart maths use integers?
- What is the difference between a signal and a computed signal?
- Why does the browser need CORS but curl does not?

If you fall behind on Day 4

Skip the register page (create the account in Swagger), skip product edit (keep list + create), and make the POS minimal: product list, add to cart, checkout. Styling can wait.

9. Day 5: finish, test, and make it yours

Goal: customers and bills in the UI, a simple dashboard, one integration test, security checks and your notes.
You will learn: integration testing, how real projects get wrapped up.

Block	What you build (names to use)	Done when
1	**Customers and bills in the UI. core/api/customers.service.ts**, **features/customers/customer-list** (search, add, edit); in the POS a customer search that shows 'earns N points'; **features/sales/sales-list** and **sale-detail** (printable bill).	A sale with a customer shows '+4 loyalty points' and the customer's balance rises on the customers page.
2	**Dashboard.** API: **Dashboard.cs** (today's sales total, bill count, low-stock list) + **DashboardController** GET summary. UI: **features/dashboard** with four cards and a low-stock table, plus **one chart** of the last 30 days' revenue (a simple bar list or SVG).	The cards change after you make a sale; the low-stock table lists a product you set low.
3	**Tests and hardening.** One **integration test** (WebApplicationFactory + SQLite): register, create a product, sell it, check stock. Add **CORS** and login **rate limiting** if not done. Run the security checklist below.	`dotnet test` and `npm test` are green; the checklist is all ticked.
4	**Wrap-up.** README (how to run), NOTES.md retrospective, compare with the reference, pick a stretch module (Section 11).	A friend could clone your repo, follow the README and run it.

Block 2: the dashboard endpoint

- **Today** means the shop's local day. Compute the UTC start and end of today for your time zone (the reference uses a StoreCalendar class and a Store:TimeZoneId setting) and filter sales by it; do not compare against UTC midnight or the numbers will be wrong for hours.
- Only count **Completed** sales. Low stock = inventory at or below its LowStockThreshold.
- For the 30-day chart, fetch the bills' `CreatedAt` and `TotalAmount`, group by local date in memory, and fill missing days with 0 so the chart has no gaps.

Block 3: the integration test pattern

```
// tests/SmartPOS.IntegrationTests
// packages: xunit, Microsoft.AspNetCore.Mvc.Testing, Microsoft.EntityFrameworkCore.Sqlite
// 1. A factory: WebApplicationFactory<Program> that swaps UseNpgsql for
//    UseSqlite("DataSource=:memory:") (keep one connection open), sets
//    Jwt:SigningKey, and calls EnsureCreated().
// 2. A helper that registers a user and returns an HttpClient with the Bearer token.
// 3. The test: create product (stock 10) -> POST /api/sales (quantity 3)
//    -> GET the product -> expect stock 7.
// Also: an oversell (quantity 99) returns 409 and stock stays 7.
```

You will need `public partial class Program;` at the bottom of Program.cs so the test project can see it.
Note: SQLite compares decimals as text, so avoid awkward discount values (use 10, not 12.5) in integration tests.

Security and quality checklist (tick every line)

- [ ] No password, connection string or signing key is in any committed file (`git grep -i password` shows only code, not values).
- [ ] `appsettings.json` has empty placeholders for secrets; real values are in user-secrets.
- [ ] Every endpoint except register and login needs a token (try one in Swagger without it: 401).
- [ ] A wrong password and an unknown email give the identical response.
- [ ] The browser never sends prices; the server recomputes the bill (try changing a price in the request: it is ignored).
- [ ] Errors shown to the user contain no stack traces or SQL.

[ ] Deleting a product that was sold is refused (409).

Day 5 retrospective (write in NOTES.md)

- Which three things did you understand best? Which three do you still not understand? (These are your next study topics.)
- Where did you copy from the reference, and could you now write it yourself?
- Which bug took longest, and what was the cause?

10. If you are behind: what to cut, in order

Falling behind is normal for a first project. Cut in **this order**, from the least valuable to the most; stop cutting as soon as the plan fits.

Cut	What	Day
1	Dashboard chart (keep the four cards)	Day 5
2	Product edit and delete (keep list + create)	Day 4
3	Register page in the UI (use Swagger to create the account)	Day 4
4	Customer edit (keep list + create)	Day 5
5	Loyalty points entirely (the sale works without a customer)	Day 3 / 5
6	The integration test (keep unit tests)	Day 5
7	Sales list page (keep the bill shown right after checkout)	Day 5

Never cut: exception handling, password hashing, JWT checks, the server-side price calculation, the calculator tests, the atomic stock update, the transaction around a sale, the CHECK constraints.

If you are ahead: do not rush on; do the 'Experiment' steps again with different inputs, read the reference code for the same module and write down every difference, then start a stretch module.

11. Stretch modules (after the 5 days)

Each is a self-contained next step with its names, in the order I would do them. Compare with the reference when done.

Module	What to build
A. Multiple shops (about 6 h)	Add a **Shop** entity and a ShopId to Users, Customers, Products, Inventories, Sales, StockMovements; put the shop id in the JWT (claim `shop`); add **ICurrentUser / ICurrentShop** and a **global query filter** in SmartPosDbContext so every query is limited to the signed-in shop; make SKU/phone unique *per shop*; registration creates the shop. Write a test with two shops proving one cannot read, change or sell the other's data. Reference: SmartPosDbContext.cs, AuthService.cs.
B. Email verification (about 5 h)	**IEmailSender**, **IEmailAddressChecker**, a development logging sender, an SMTP sender; a single-use token (store only its SHA-256 hash, 24-hour expiry); endpoints verify-email and resend-verification; a banner in the shell and a verify-email page. Reject throwaway-mailbox domains. Reference: Auth/AuthService.cs, Infrastructure/Email/EmailServices.cs.
C. The insights dashboard (about 8 h)	**Insights.cs** + GET /api/dashboard/insights; **shared/charts**: KpiCard, CountUp, Sparkline, AreaChart, BarChart, HBarChart, DonutChart, Heatmap. Reference: Application/Dashboard/Insights.cs and frontend/src/app/shared/charts/.
D. Demo data and tools (about 4 h)	**SampleData** + **SampleDataSeeder** (a fictional catalogue for each new account); a seeded generator of fabricated sales history; a development-only command to reset a password. Reference: Infrastructure/Persistence/DemoHistory.cs, ResetPasswordCommand.cs.
E. Exact line splits (about 2 h)	Split the bill's discount and tax across lines to the cent with the largest-remainder method so SaleItem discounts/taxes add up exactly. Reference: SaleCalculator.Allocate.
F. Cancel and refund (about 4 h)	Not in the reference yet: a cancel endpoint that sets Status to Cancelled, restocks with a new movement type, and reverses loyalty points. A good test of everything you learned.

12. Cheat sheets

Commands

```
# backend (from smartpos/backend)
dotnet build                                  dotnet test
dotnet run --project src/SmartPOS.API --launch-profile http      # http://localhost:5080/swagger
dotnet ef migrations add NAME --project src/SmartPOS.Infrastructure \
    --startup-project src/SmartPOS.API --output-dir Persistence/Migrations
dotnet ef database update --project src/SmartPOS.Infrastructure --startup-project src/SmartPOS.API
dotnet user-secrets list --project src/SmartPOS.API

# frontend (from smartpos/frontend/smartpos-web)
npm start        npm test        npm run build

# database
psql smartpos_dev -c "\dt"          psql smartpos_dev -c "select * from \"Products\" limit 5"
```

Common errors and what they mean

You see	Usually means
Address already in use (port 5080)	Another copy of the API is running. Stop it (Ctrl+C in its terminal, or find the process).
'dotnet' not found	Add ~/.dotnet to PATH in that terminal.
Blocked by CORS policy	The API has no CORS policy for http://localhost:4200, or UseCors is after authentication.
401 on every call	The token is missing (interceptor), expired, or the signing key differs from the one that issued it.
Column "price" does not exist (migration)	A CHECK constraint used an unquoted column name. Quote it inside the SQL string.
No such table / pending migrations	You forgot dotnet ef database update after adding a migration.
The LINQ expression could not be translated	EF cannot turn that C# into SQL. Move the C# method call out of the query, or fetch first then compute.
Cannot access a disposed context / lazy errors	A query ran after the request ended. await every async call.
Every test passes but the real app fails	SQLite in tests differs from PostgreSQL (case-sensitivity, decimals). Test the real thing in Swagger too.

Where things are in the reference project

Topic	Path
Entities	backend/src/SmartPOS.Core/Entities/
DbContext and configurations	backend/src/SmartPOS.Infrastructure/Persistence/
Sale rules	backend/src/SmartPOS.Application/Sales/ (SaleCalculator.cs, SaleService.cs)
Stock update	SmartPosDbContext.cs (TryChangeStockAsync) and Inventory/StockRules.cs
Loyalty	backend/src/SmartPOS.Application/Customers/Loyalty.cs
Auth	Application/Auth/AuthService.cs, API/Auth/JwtTokenService.cs, API/Infrastructure/ApiServiceCollectionExtensions.cs
Frontend core	frontend/src/app/core/ (auth, api, models.ts)
Cashier screen	frontend/src/app/features/pos/ (cart.ts, pos.ts, pos.html)
Other docs	README.md, docs/ARCHITECTURE.md, docs/DATABASE.md, docs/API.md, docs/SmartPOS-Project-Guide.pdf

13. Glossary and where to learn more

Term	Meaning
API	The set of URLs your server offers to the browser.
DTO	Data Transfer Object: a small class describing exactly what goes over HTTP.
Entity	A class that is stored as a database table.
Migration	A versioned script that changes the database structure.
Transaction	Changes that all happen or none do.
Atomic	Done in one indivisible step; nothing can happen in between.
Snapshot	A copy of a value at a moment in time (the price on a bill).
JWT	A signed token proving who you are.
Hash	A one-way fingerprint of a value (used for passwords).
Signal	An Angular value that updates the screen when it changes.
Interceptor	Code that runs on every HTTP call.
CORS	A browser rule about which sites may call which servers.
Idempotent	Safe to repeat: doing it twice has the same effect as once.
CHECK constraint	A database rule on a column's values.

Free official learning material (use when a block feels like magic)

- **Microsoft Learn** (learn.microsoft.com): 'Create a web API with ASP.NET Core', 'Entity Framework Core' getting started, 'Authentication and authorization in ASP.NET Core'.
- **Angular** (angular.dev): the official tutorials ('Learn Angular') on components, signals, routing and forms.
- **PostgreSQL documentation** (postgresql.org/docs): the tutorial chapter, and 'Constraints'.
- **This repository:** docs/ARCHITECTURE.md explains the decisions; docs/SmartPOS-Project-Guide.pdf walks through every file of the finished app.

A last word

You will get stuck, and that is the point: the stuck minutes are where the learning happens. Keep each block small, keep every check honest, and when the five days are over, **you will own a project you can explain line by line**, which is worth far more than one you only ran.